

Editing the screens

Everything a customer sees lives in three files in `renderer/`. Nothing else in the project needs touching to change how the booth looks or reads.

File	What it holds
<code>renderer/index.html</code>	The screens, and every word on them
<code>renderer/styles.css</code>	Colours, type, spacing, layout
<code>renderer/app.js</code>	Text that is built from data (prices, counts, the code)

To see a change: **save the file, then press `Cmd+R` with the booth focused.** No build step, no restart. (Changing `main.js`, `server.js` or `config.js` does need a restart — those are not UI.)

Run the booth in test mode while you work:

```
npm run mock
```

That gives you generated photos, no camera, and nothing actually prints.

1. Changing words

Open `renderer/index.html` and search for the words you want to change. Each screen is a `<section>` marked with the name the app uses:

```
<section class="screen screen-ready" data-screen="ready">
```

The screens, in the order a customer meets them:

data-screen	What it is
<code>welcome</code>	The layout menu. Tapping a card starts a session
<code>ready</code>	Get ready. Live view cropped to the chosen shape
<code>pose</code>	The shoot, with the countdown and the filmstrip

data-screen	What it is
pick	Review 1 of 3 — choose photos
frame	Review 2 of 3 — choose a frame
filter	Review 3 of 3 — choose a look
printing	The moment between finishing and getting a code
done	The pickup code. The booth rests here
locked	Waiting for staff to start the next group

Some text is **not** in the HTML, because it is built from settings or from what the customer chose. That lives in `renderer/app.js` — search for the wording you see on screen:

On screen	Where
\$5, Grand, 4 on the print · 1 strip	<code>buildMenu()</code>
Decoration names on the frame step	<code>buildDecorChips()</code>
Photo 3 of 10	<code>runCaptureSequence()</code>
Nice. Get set for the next one.	<code>runCaptureSequence()</code>
2 of 4 chosen	<code>buildThumbs()</code>
The pickup code and its bill	<code>renderTicket()</code>

Prices, layout names and notes are **not** code — they are in `settings.json` under `pricing`. Change them there.

2. Changing colours

All of them are at the top of `renderer/styles.css`, in `:root`. Change one value and it changes everywhere it is used:

```
:root {
  --ink: #26357e;      /* the blue: text, lines, buttons */
  --ink-soft: #6b77ae; /* quieter text: hints, captions */
  --ink-line: rgba(38, 53, 126, 0.3); /* dashed rules, borders */
  --paper: #fff8ee;     /* the cream card */
  --paper-2: #f7ecd9;   /* pressed/inset paper */
  --stage-1: #3b4d97;   /* blue background behind the card */
  --stage-2: #1d2a68;
  --seal: #8b0003;      /* the wax seal, and warnings */
}
```

Do not hard-code a colour anywhere else. If you find yourself typing `#26357e` into a rule, use `var(--ink)` instead — otherwise the next person changing the palette will miss it.

Frame designs are not code either. Most of what a guest picks between on the frame step is the designer's artwork in `frames/designs/`: a whole frame with transparent holes where the photos go. They come from the designer's PNGs through an importer, which finds every hole and works out the photo slots, so nobody types coordinates:

```
uv run --with pillow --with numpy --with scipy scripts/import-designs.py ~/Downloads
```

Point it at the folder holding the designer's `strip-frame 3` (Trio), `strip-frame 4` (Quad) and `bigframe 4` (Grand) folders. Holes that are shapes rather than rectangles (hearts, ovals) are detected and show the whole photo inside.

Decorations only appear on the plain frame. They are the art a customer picks between when they choose no design, and they are PNG files in `frames/decor/`. Drop a PNG in that folder and it appears as a new option next time the booth starts — no edits anywhere.

Draw them **564 × 1764** (one strip column, 300dpi) with a **transparent middle**, or they will cover the photographs. A two-column layout gets the same art over each column, since those get cut apart and handed to two people.

`frames/decor/decor.json` is optional and only sets display names and order:

```
[{ "file": "ribbon.png", "name": "Ribbon" }]
```

To get a correctly sized, correctly transparent file to start from:

```
node scripts/make-decor.js
```

That writes five placeholder designs. Open one in a drawing app and paint over it. Decorations draw *under* the frame's caption, so the wordmark survives; if your art has its own caption, blank `caption` for that layout in `frames/frames.json`.

3. Changing sizes and spacing

Type sizes and gaps are also `:root` variables, and they are all `clamp()`:

```
--fs-md: clamp(15px, 2.3vmin, 24px);  
/*           ^min   ^scales   ^max */
```

The middle value scales with the screen, so the booth reads the same on the 1080×1920 panel and on a laptop. To make something bigger everywhere, raise its `--fs-*`. To make one thing bigger, change that rule:

Want	Rule
Bigger prices on the menu	<code>.menu-price</code>
Bigger pickup code	<code>.ticket-code</code>
Bigger countdown	<code>.countdown .ring</code> and <code>.countdown-num</code>
Wider cards	<code>.paper { max-width: ... }</code>

Two rules the layout depends on. Break either and things overlap:

- 1 Any flexible area needs `min-height: 0`, or it refuses to shrink and pushes its neighbours off screen.
- 2 Let *one* flexible thing absorb the slack, and make it an image or an SVG rather than text. Text that shrinks becomes unreadable; a drawing just gets smaller. This is why the welcome cards flex on their glyph.

The flow test measures this. If you make something overlap, `node scripts/e2e.js` will say so rather than you finding out at the event.

4. Common edits

Move something on a screen. Find the `<section>` in `index.html`. Most screens are a header, a middle, and a footer:

```
<div class="cut cut-poster">  
  <header class="poster-head"> ... </header>  
  <div class="menu-cards" id="menuList"></div>  <!-- the flexible middle -->  
  <footer class="poster-foot"> ... </footer>  
</div>
```

Moving a line between header and footer is usually all that is needed.

Add a line of text. Copy an existing one so it inherits a style:

```
<p class="menu-fine">Ask staff about keychains.</p>
```

Useful classes: `.poster-lead` (the instruction), `.poster-cta` (quieter sentence), `.menu-fine` (small print), `.ready-warn` (red warning).

Change a button's words. In `index.html`, by its `id`. Keep the `id` — the JavaScript finds the button by it, and the flow test checks some of them.

Add a photo layout. Not a UI change — add it to `frames/frames.json` and price it in `settings.json`. The menu card, its little diagram, and the crop guide are all generated from the geometry, so they cannot disagree with what prints.

5. Before you finish

```
npm run check          # a few seconds, no display needed
node scripts/e2e.js    # about a minute, drives a whole session
```

The flow test catches the two things that are easy to break by accident: elements overlapping each other, and a button whose `id` the JavaScript still expects. It restores your `settings.json` afterwards.

If a screen goes blank after an edit, open the console — `View ▶ Toggle Developer Tools`, or `Cmd+Option+I`. A renamed or deleted `id` is nearly always the cause, and the error names it.